TensorFlow: Qwik Start

experimentLabschedule45 minutesuniversal_currency_alt1 Creditshow_chartIntroductory

Rate Lab

infoThis lab may incorporate AI tools to support your learning.

GSP637

Overview

In this lab you will learn the basic ‘Hello World' of machine learning where, instead of programming explicit rules in a language such as Java or C++, you build a system that is trained on data to infer the rules that determine a relationship between numbers.

Objectives

In this lab, you will learn how to:

- Set up the development environment in the Jupyter notebook
- Design a machine learning model
- Train a neural network
- Test a model

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources are made available to you.

This hands-on lab lets you do the lab activities in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials you use to sign in and access Google Cloud for the duration of the lab.

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).

Note: Use an Incognito (recommended) or private browser window to run this lab. This prevents conflicts between your personal account and the student account, which may cause extra charges incurred to your personal account.

- Time to complete the lab—remember, once you start, you cannot pause a lab.

Note: Use only the student account for this lab. If you use a different Google Cloud account, you may incur charges to that account.

Introduction

Consider the following problem: you're building a system that performs activity recognition for fitness tracking. You might have access to the speed at which a person is moving and attempt to infer their activity based on this speed using a conditional:

```
if(speed<4){
  status=WALKING;
}
```

- You could extend this to *running* with another condition:

```
if(speed<4){
    status=WALKING;
} else {
    status=RUNNING;
}
```

- Similarly, you could detect *cycling* with another condition:

```
if(speed<4){
    status=WALKING;
} else if(speed<12){
    status=RUNNING;
} else {
    status=BIKING;
}
```

- Now consider what happens when you want to include an activity like *golf*? Now, it becomes less obvious how to create a rule to determine the activity.

```
// Now what?
```

It's extremely difficult to write a program (expressed in code) that helps you detect the golfing activity.

So what do you do? You can use machine learning to solve the problem!

What is machine learning?

In the previous section you encountered a problem when you tried to determine a user's fitness activity. You hit limitations in what you could achieve by writing more code since your conditions have to be more complex to detect an activity like **golf**.

Consider building applications in the traditional manner as represented in the following diagram:

You express **rules** in a programming language. These act on **data** and your program provides **answers**. In the case of activity detection, the rules (the code you wrote to define types of activities) acted upon the data (the person's movement speed) in order to find an answer -- the return value from the function for determining the activity status of the user (whether they were walking, running, biking, etc.).

The process for detecting this activity via machine learning is very similar -- only the axes are different:

Instead of trying to define the rules and expressing them in a programming language, you provide the answers (typically called labels) along with the data. The machine then infers the rules that determine the relationship between the answers and the data. For example, in a machine learning context, your activity detection scenario might look like this:

```
0101001010100101010
1001010101001011101
0100101010010101001
0101001010100101010
```

Label = WALKING

```
1010100101001010101
0101010010010010001
0010011111010101111
1010100100111101011
```

Label = RUNNING

```
1001010011111010101
1101010111010101110
1010101111010101011
1111110001111010101
```

Label = BIKING

```
1111111111010011101
0011111010111110101
0101110101010101110
1010101010100111110
```

Label = GOLFING
(Sort of)

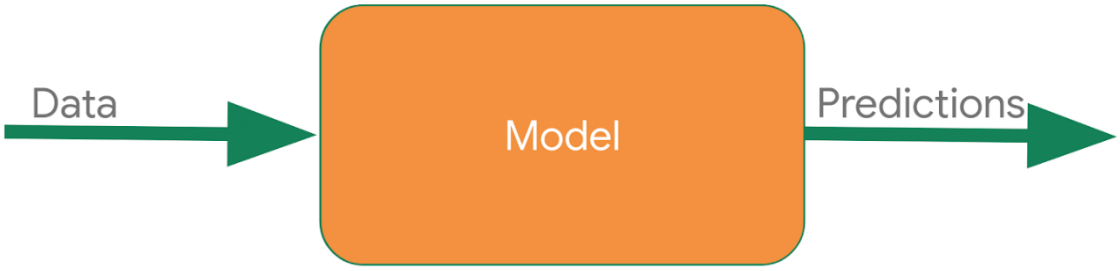
You gather lots of data, and label it to effectively say "This is what walking looks like", "This is what running looks like" etc. Then, from the data, the computer can infer the rules that determine what the distinct patterns that denote a particular activity are.

Beyond being an alternative method to programming this scenario, this also gives you the ability to open up new scenarios, such as golfing, which may not have been possible under the traditional rule-based programming approach.

In traditional programming your code compiles into a binary that is typically called a program. In machine learning, the item that you create from the data and labels is called a model.

So if you go back to this diagram:

Consider the result of the above to be a model, which is used like this at runtime:



You pass the model some data, and the model uses the rules it inferred from the training to come up with a prediction -- i.e. "That data looks like walking", "That data looks like biking" etc.

In this lab you will build a very simple 'Hello World' model made up of the building blocks that can be used in any machine learning scenario!

Task 1. Open the notebook in Vertex AI Workbench

1. In the Google Cloud console, on the **Navigation menu** (☰), click **Vertex AI > Workbench**.

2. Find the `Workbench instance name` instance and click on the **Open JupyterLab** button.

The JupyterLab interface for your Workbench instance opens in a new browser tab.

Install TensorFlow and additional packages

1. From the Launcher menu, under **Other**, select **Terminal**.

2. Check if your **Python** environment is already configured. Copy and paste the following command in the terminal.

```
python --version
```
Copied!

content_copy

Example output:

```
Python 3.10.14
```

3. Run the following command to install the **TensorFlow** package.

```
pip3 install tensorflow
```
Copied!

content_copy

4. To upgrade `pip3`, run the following command in the terminal.

```
pip3 install --upgrade pip
```
Copied!

content_copy

Pylint is a tool that checks for errors in Python code, and highlights syntactical and stylistic problems in your Python source code.

5. Run the following command to install the `pylint` package.

```
pip install -U pylint --user
```
Copied!

content_copy

6. Install the packages required for the lab in the `requirements.txt` file:

```
pip install -r requirements.txt
```
Copied!

content_copy

Now, your environment is set up!

Task 2. Create your first machine learning model

Consider the following sets of numbers. Can you see the relationship between them?

X:	-1	0	1	2	3	4
Y:	-2	1	4	7	10	13

As you read left to right, notice that the X value is increasing by 1 and the corresponding Y value is increasing by 3. So, the relationship should be **Y=3X** plus or minus some value.

Then, take look at the 0 on X and see that the corresponding Y value is 1.

From both of these observations, you can determine that the relationship is **Y=3X+1**.

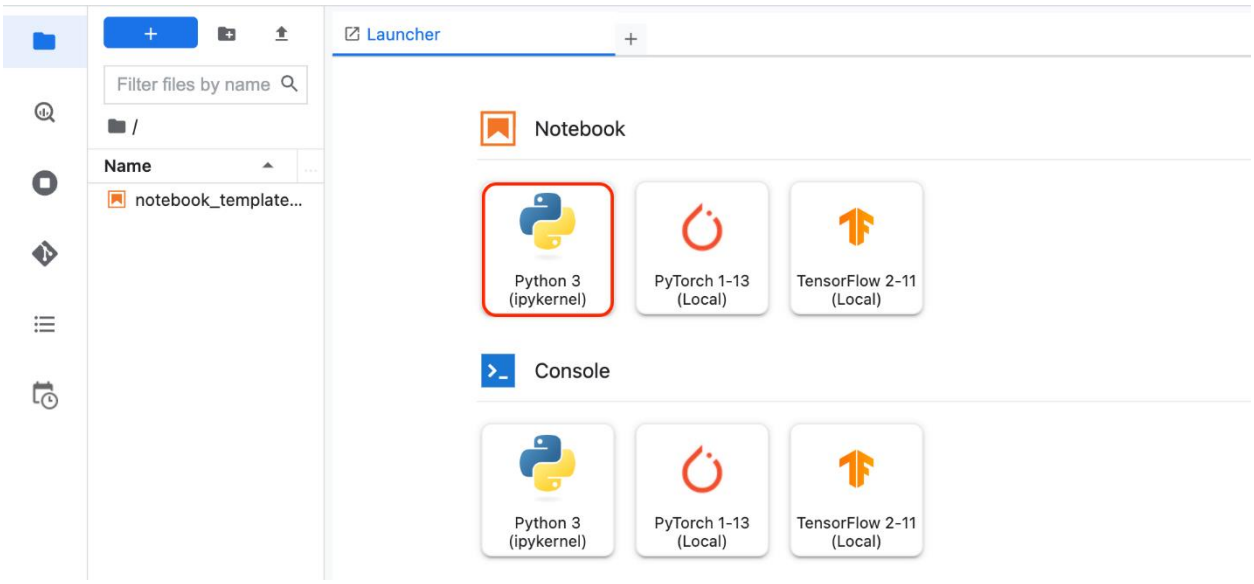
This is almost exactly how you would use code to train a model, known as a **neural network**, to spot the patterns in the data!

You use data to train the neural network! By feeding it with a set of Xs and a set of Ys, it should be able to figure out the relationship between them.

Create a new notebook and import libraries

1. Click the **+** icon on the left side of the Workbench to open a new Launcher.

2. From the Launcher menu, under **Notebook**, select **Python3**.



You will be presented with a new Jupyter notebook. For more information on how to use Jupyter notebooks, see the Jupyter Notebook documentation.

3. Import and configure `logging` and `google-cloud-logging` for Cloud Logging. **In the first cell**, add the following code:

```
import logging
import google.cloud.logging as cloud_logging
from google.cloud.logging.handlers import CloudLoggingHandler
from google.cloud.logging_v2.handlers import setup_logging

cloud_logger = logging.getLogger('cloudLogger')
cloud_logger.setLevel(logging.INFO)
cloud_logger.addHandler(CloudLoggingHandler(cloud_logging.Client()))
cloud_logger.addHandler(logging.StreamHandler())
```

Copied!

content_copy

4. Import `tensorflow` for training and evaluating the model. Call it `tf` for ease of use. Add the following code to the first cell.

```
# Import TensorFlow
import tensorflow as tf
```

Copied!

content_copy

5. Import `numpy`, to parse through the data for debugging purposes. Call it `np` for ease of use. Add the following code to the first cell.

```
# Import numpy
import numpy as np
```

Copied!

content_copy

6. To run the cell, either click the **Run** button or press **Shift + Enter**.

7. Save the notebook. Click **File -> Save**. Name the file `model.ipynb` and click **OK**.

Note: you can ignore any warnings that may appear in the output.

Prepare the data

Next up, you will prepare the data your model will be trained on. In this lab, you're using the 6 Xs and 6 Ys used earlier:

X:	-1	0	1	2	3	4
Y:	-2	1	4	7	10	13

As you can see, the relationship between the Xs and Ys is **Y=3x+1**, so where X = 1, Y = 4 and so on.

A python library called `numpy` provides lots of array type data structures that are a defacto standard way of feeding in data. To use these, specify the values as an array in `numpy` using `np.array([])`

1. In the **second cell**, add the following code:

```
xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
ys = np.array([-2.0, 1.0, 4.0, 7.0, 10.0, 13.0], dtype=float)
```

Copied!

content_copy

Design the model

In this section, you will design your model using TensorFlow.

You will use a machine learning algorithm called neural network to train your model. You will create the simplest possible neural network. It has 1 layer, and that layer has 1 neuron. The neural network's input is only one value at a time. Hence, the input shape must be [1].

Note: You will learn more about neural networks in the upcoming labs in this quest.

1. In the **second cell**, add the following code:

```
model = tf.keras.Sequential([tf.keras.layers.Dense(units=1,
input_shape=[1])])
```
Copied!

content_copy

Compile the model

Next, you will write the code to compile your neural network. When you do, you must specify 2 functions, a `loss` and an `optimizer`.

If you've seen lots of math for machine learning, this is where you would usually use it, but `tf.keras` nicely encapsulates it in functions for you.

- From your previous examination, you know that the relationship between the numbers is `y=3x+1`.

- When the computer is trying to *learn* this relationship, it makes a guess...maybe `y=10x+10`. The `loss` function measures the guessed answers against the known correct answers and measures how well or how badly it did.

Note: Learn more about different types of loss functions available in `tf.keras` from the Module: tf.keras.losses documentation.

- Next, the model uses the optimizer function to make another guess. Based on the loss function's result, it will try to minimize the loss. At this point, maybe it will come up with something like `y=5x+5`. While this is still pretty bad, it's closer to the correct result (i.e. the loss is lower).

Note: Learn more about different types of optimizers available in `tf.keras` from the Module: tf.keras.optimizers documentation.

- The model repeats this for the number of epochs you specify.

1. Add the following code to the **second cell**:

```
model.compile(optimizer=tf.keras.optimizers.SGD(),
loss=tf.keras.losses.MeanSquaredError())
```
Copied!

content_copy

In the above code snippet, you tell the model to use `mean_squared_error` for the loss and `stochastic gradient descent (sgd)` for the optimizer. You don't need to understand the math for these yet, but you will see that they work!

Note: Over time you will learn the appropriate loss and optimizer functions for different scenarios.

Train the neural network

To train the neural network to 'learn' the relationship between the Xs and Ys, you will use `model.fit`.

This function will train the model in a loop where it will make a guess, measure how good or bad it is (aka the loss), use the optimizer to make another guess, etc. It will repeat this process for the number of epochs you specify, which in this lab is 500.

1. Add the following code to the **second cell**:

```
model.fit(xs, ys, epochs=500)
```

Copied!

content_copy

In the above code `model.fit` will train the model for a fixed number of epochs.

Note: Learn more about `model.fit` from the fit section of the tf.keras.Model documentation.

Now, your file should look like this (note that the code will be in two separate cells):

```
import logging
import google.cloud.logging as cloud_logging
from google.cloud.logging.handlers import CloudLoggingHandler
from google.cloud.logging_v2.handlers import setup_logging

cloud_logger = logging.getLogger('cloudLogger')
cloud_logger.setLevel(logging.INFO)
cloud_logger.addHandler(CloudLoggingHandler(cloud_logging.Client()))
cloud_logger.addHandler(logging.StreamHandler())

import tensorflow as tf
import numpy as np

xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
ys = np.array([-2.0, 1.0, 4.0, 7.0, 10.0, 13.0], dtype=float)

model = tf.keras.Sequential([tf.keras.layers.Dense(units=1,
input_shape=[1])])

model.compile(optimizer=tf.keras.optimizers.SGD(),
loss=tf.keras.losses.MeanSquaredError())

model.fit(xs, ys, epochs=500)
```

Copied!

content_copy

Run the code

Your script is ready! Run it to see what happens.

1. Click the **Run** button or press **Shift + Enter** to run the second cell in the notebook.

2. Look at the output. Notice that the script prints out the loss for each epoch. Your output may be slightly different that what is illustrated here.

Note: A number with `e-` in the value is being displayed in scientific notation with a negative exponent.

If you scroll through the epochs, you see that the loss value is quite large for the first few epochs, but gets smaller with each step. For example:

```
Epoch 2/500
6/6 [==============================] - 0s 242us/sample - loss: 52.1992
Epoch 3/500
6/6 [==============================] - 0s 192us/sample - loss: 41.0681
Epoch 4/500
6/6 [==============================] - 0s 153us/sample - loss: 32.3107
Epoch 5/500
6/6 [==============================] - 0s 142us/sample - loss: 25.4207
Epoch 6/500
6/6 [==============================] - 0s 142us/sample - loss: 20.0001
Epoch 7/500
6/6 [==============================] - 0s 146us/sample - loss: 15.7354
Epoch 8/500
6/6 [==============================] - 0s 141us/sample - loss: 12.3801
Epoch 9/500
6/6 [==============================] - 0s 140us/sample - loss: 9.7403
```

As the training progresses, the loss gets very small:

```
Epoch 45/500
6/6 [==============================] - 0s 146us/sample - loss: 0.0023
Epoch 46/500
6/6 [==============================] - 0s 154us/sample - loss: 0.0020
Epoch 47/500
6/6 [==============================] - 0s 145us/sample - loss: 0.0017
Epoch 48/500
6/6 [==============================] - 0s 139us/sample - loss: 0.0014
Epoch 49/500
6/6 [==============================] - 0s 149us/sample - loss: 0.0012
Epoch 50/500
6/6 [==============================] - 0s 178us/sample - loss: 0.0011
Epoch 51/500
6/6 [==============================] - 0s 142us/sample - loss: 9.5416e-04
Epoch 52/500
6/6 [==============================] - 0s 149us/sample - loss: 8.5538e-04
Epoch 53/500
6/6 [==============================] - 0s 145us/sample - loss: 7.7552e-04
Epoch 54/500
6/6 [==============================] - 0s 145us/sample - loss: 7.1057e-04
```

And by the time the training is done, the loss becomes extremely small, showing that our model is doing a great job of inferring the relationship between the numbers:

```
Epoch 495/500
6/6 [==============================] - 0s 150us/sample - loss: 5.4194e-08
Epoch 496/500
6/6 [==============================] - 0s 149us/sample - loss: 5.3076e-08
Epoch 497/500
6/6 [==============================] - 0s 154us/sample - loss: 5.1974e-08
Epoch 498/500
6/6 [==============================] - 0s 145us/sample - loss: 5.0963e-08
Epoch 499/500
6/6 [==============================] - 0s 152us/sample - loss: 4.9884e-08
Epoch 500/500
6/6 [==============================] - 0s 139us/sample - loss: 4.8896e-08
```

You probably don't need all 500 epochs, try experimenting with different values. Looking at this example, the loss is really small after only 50 epochs, so that might be enough!

Click *Check my progress* to verify the objective.

Create machine learning models

Check my progress

Using the model

You now have a model that has been trained to learn the relationship between X and Y.

You can use the `model.predict` method to figure out the Y for an X not previously seen by the model during training. So, for example, if X = 10, what do you think Y will be?

1. Add the following code to the **third cell** to make a prediction:

```
cloud_logger.info(str(model.predict(np.array([10.0]))))
```

Copied!

content_copy

Note: Your prediction result is passed to `cloud_logger` in order to produce cloud logs which can be checked for progress.

2. Press **Ctrl+s** or click **File** -> **Save** to save your notebook.

3. To run the third cell, either click the **Run** button or press **Shift + Enter**.

The Y value is listed after the training log (epochs).

Example output:

```
.
.
.
Epoch 498/500
6/6 [==============================] - 0s 175us/sample - loss: 4.2870e-06
Epoch 499/500
6/6 [==============================] - 0s 158us/sample - loss: 4.1995e-06
Epoch 500/500
6/6 [==============================] - 0s 152us/sample - loss: 4.1129e-06
[[31.005917]]
```

You might have thought Y=31, right? But it ended up being a little over (31.005917). Why do you think that is?

Answer: Neural networks deal with *probabilities*. It calculated that there is a very high probability that the relationship between X and Y is Y=3X+1. But with only 6 data points it can't know for sure. As a result, the result for 10 is very close to 31, but not necessarily 31.

As you work with neural networks, you'll see this pattern recurring. You will almost always deal with probabilities, not certainties, and will do a little bit of coding to figure out what the result is based on the probabilities, particularly when it comes to classification.

Click *Check my progress* to verify the objective.

Use the model

Check my progress

Congratulations!

Congratulations! In this lab, you have created, trained, and tested your own Machine Learning Model using TensorFlow.

Next steps / learn more

- See all there is to know about Vertex AI.
- Fun stuff with AI! See Experiments with Google.
- Learn more about TensorFlow.
- Bracketology with Google Machine Learning
- Getting Started with BQML
- Predict Taxi Fare with a BigQuery ML Forecasting Model

Google Cloud training and certification

...helps you make the most of Google Cloud technologies. Our classes include technical skills and best practices to help you get up to speed quickly and continue your learning journey. We offer fundamental to advanced level training, with on-demand, live, and virtual options to suit your busy schedule. Certifications help you validate and prove your skill and expertise in Google Cloud technologies.

Manual Last Updated September 16, 2024

Lab Last Tested September 16, 2024